ASSIGNMENT 2

Software Testing and Quality Assurance

MCA 2nd Year

Questions 1–15: Detailed Answers

Prepared for academic submission

Q1. Develop a complete testing strategy for an Online Examination Management System.

An Online Examination Management System allows students to register for examinations, select subjects, make payments, download admit cards, and view results. Because it handles personal information, financial transactions, and academic records, it must be accurate, secure, reliable, and usable.

Testing fundamentals:
• Testing reveals the presence of defects but cannot prove that software is completely defect-free.
• Exhaustive testing is impractical, so testing should prioritize high-risk functions such as payment and result calculation.
• Early testing reduces the cost of correcting requirement and design defects.
• Defects often cluster in complex modules such as registration, payment, and result processing.
• Testing must be context-dependent; confidentiality and academic accuracy are especially important here.
• The absence of visible errors does not mean the system meets user needs.

Testing types and their application:
• Unit testing: test registration validation, fee calculation, grade assignment, and other functions individually.
• Integration testing: verify communication among registration, payment gateway, database, and admit-card services.
• System testing: test complete workflows from registration through result publication.
• Acceptance testing: university staff and selected students verify realistic workflows.
• Functional testing: check registration, subject selection, payments, admit cards, and results.
• Performance testing: simulate many simultaneous registrations or result checks.
• Security testing: test authentication, authorization, SQL injection, and access to another student's records.
• Usability testing: assess navigation, understandable error messages, and accessibility.
• Regression testing: rerun existing tests after changes to fees, registration, or results.
• Recovery testing: verify safe recovery after server crashes or interrupted payments.

Static testing examines requirements, designs, code, and test cases without executing the application. Reviewers should identify ambiguous registration deadlines, missing payment-failure rules, poor database relationships, insecure password handling, and incomplete test coverage. Static-analysis tools can flag unused variables, coding-standard violations, and potential vulnerabilities.

Dynamic testing executes the system with test data. Examples include registering a valid student, rejecting invalid subject selections, simulating successful and failed payments, checking admit-card details, validating calculated marks, and attempting unauthorized access to results.

Verification asks, “Are we building the product right?” It checks conformance to specifications through requirement reviews, design inspections, code reviews, and standards checks. Validation asks, “Are we building the right product?” It confirms that students and administrators can complete real tasks successfully through system, usability, and acceptance testing.

Testing across development stages:
1. Requirement analysis: review requirements, identify ambiguity, assess risk, and prepare a requirements traceability matrix.
2. System design: inspect architecture, database relationships, payment integration, and security design.
3. Implementation: conduct peer reviews, static analysis, and unit testing.
4. Integration/system testing: test end-to-end workflows, security, performance, compatibility, and recovery.
5. Acceptance/release: resolve critical defects, run regression tests, obtain university approval, and deploy.

Conclusion: Testing should begin during requirement analysis and continue through deployment. Static testing prevents defects early, dynamic testing checks execution, verification checks specifications, and validation checks user needs. Together these activities improve accuracy, confidentiality, and reliability.

Q2. Identify defects detectable through static testing in an Online Banking Application and explain cost reduction.

An online banking application supports login, money transfers, beneficiaries, bill payments, and transaction history. Static testing examines requirements, designs, source code, and other artifacts without running the application.

Possible requirement defects:
1. Ambiguous transfer limits, such as “reasonable amount” without a numeric maximum.
2. Incomplete beneficiary rules, including whether newly added beneficiaries can be used immediately.
3. Missing requirements for a payment timeout after money has been debited.
4. Contradictory rules about banking hours or 24-hour transfers.
5. Unspecified transaction fees or tax treatment.

Possible design defects:
6. A workflow that debits the sender without ensuring the recipient is credited.
7. Missing concurrency control, allowing simultaneous withdrawals to use the same balance.
8. Incorrect database relationships among sender, recipient, and transaction records.
9. Inadequate audit information for investigating disputed transactions.
10. Missing recovery and reconciliation design for interrupted transfers.

Possible source-code defects:
11. Incorrect balance arithmetic or use of the wrong variable.
12. Missing validation for zero, negative, or excessively precise transfer amounts.
13. Hard-coded passwords, tokens, or API secrets.
14. SQL injection caused by concatenating user input into queries.
15. Insecure password storage or comparison.
16. Missing authorization checks that could expose another customer's account.
17. Ignored exceptions that leave partially completed transactions.
18. Duplicated transfer logic that behaves inconsistently.
19. Database connections or streams that are not released.
20. Missing logs for failed transfers or suspicious login attempts.

Illustrative pseudocode:

```
public void transfer(long fromAccount, long toAccount, double amount) {
Account sender = accountRepository.findById(fromAccount);
sender.setBalance(sender.getBalance() - amount);
accountRepository.save(sender);
Account receiver = accountRepository.findById(toAccount);
receiver.setBalance(receiver.getBalance() + amount);
accountRepository.save(receiver);
}
```

A review reveals no positive-amount validation, no sufficient-funds check, no recipient eligibility check, no explicit atomic transaction, possible race conditions, and use of double for money. A safer implementation uses decimal arithmetic, authorization, transaction boundaries, concurrency control, and consistent failure handling.

Static testing reduces later cost because requirement review prevents test design based on unclear rules; design review finds architectural problems before integration; code inspection catches defects before runtime; security analysis identifies common vulnerabilities before penetration testing; and traceability review ensures requirements have tests. An error found in a requirement may take minutes to correct, while the same error found after release may require code, database, integration, and customer-support changes.

Static testing cannot establish runtime performance or reveal every deployment, network, and concurrency failure. Dynamic testing remains necessary. Conclusion: static testing finds many defects early and lets dynamic testing focus on runtime behavior, integration, security, and realistic transactions.

Q3. Design a formal review process for a 2,000-line module.

A formal review is a structured examination of a software artifact against requirements, design rules, coding standards, and quality objectives. For a 2,000-line module with inconsistent naming, duplicated logic, inadequate comments, and complex conditions, the team should use a documented review process.

Objectives:
- Detect defects before execution or integration.
- Improve readability and consistency.
- Identify duplication and safe refactoring opportunities.
- Reduce conditional complexity.
- Check coding-standard compliance and documentation.
- Record corrective actions and verification results.

Participants:
- Moderator: plans the review and controls the meeting.
- Author: explains the module and corrects findings.
- Reviewers: inspect code independently and identify defects.
- Recorder/scribe: records findings, severity, locations, and decisions.
- Technical lead: checks architecture and approves important changes.
- QA engineer: checks testability, traceability, and regression risks.

Preparation:
1. Define scope, dependencies, interfaces, and relevant requirements.
2. Ensure the code version is correct and basic checks have completed.
3. Distribute source code, design specifications, standards, and tests.
4. Divide the module into logical classes, methods, or sections.
5. Assign reviewer responsibilities.
6. Set objectives for naming, duplication, complexity, error handling, comments, and security.
7. Ask reviewers to inspect independently and record findings.
8. Set a reasonable review rate to avoid fatigue.

Stages:
1. Planning: establish scope, roles, schedule, and entry criteria.
2. Individual preparation: inspect code and record suspected defects.
3. Review meeting: discuss findings and agree on actions.
4. Rework: author corrects accepted defects.
5. Follow-up: reviewers verify corrections and QA reruns relevant tests.
6. Closure: record unresolved risks and confirm exit criteria.

Checklist:
- Are class, method, variable, and constant names meaningful and consistent?
- Does each method have a clear responsibility?
- Are long methods divided where appropriate?
- Is logic duplicated unnecessarily?
- Are nested conditions understandable and correct?
- Are null values and boundaries handled?
- Are arithmetic and logical operators correct?
- Are exceptions handled and resources released?
- Are comments accurate and useful rather than merely repeating code?
- Is user input validated and sensitive data protected?
- Are authorization checks present?
- Are unnecessary database calls avoided?
- Are important behaviors covered by tests?

Defect classification:
- Critical: severe security, data-integrity, or system-wide failure.
- Major: incorrect behavior or substantial reliability problem.
- Moderate: localized defect or significant maintenance issue.

• Minor: limited-impact readability or style issue.
• Observation: optional improvement rather than a confirmed defect.

Documentation should include a unique ID, file/class/method and line range, description, severity, expected behavior, corrective action, owner, status, verification evidence, and closure date. Example: REV-014 identifies duplicated validation logic in two methods; the team evaluates a shared method and runs regression tests after refactoring.

Follow-up: the author fixes findings, the moderator checks required actions, reviewers recheck significant changes, unit and regression tests run, accepted exceptions are documented, and the review closes only when exit criteria are met.

Conclusion: formal reviews combine preparation, independent inspection, structured discussion, documented findings, correction, and verification. They improve maintainability and reduce the risk of defects surviving into production.

Q4. Difference between peer review and formal review; peer-review procedure for an E-Commerce payment module.

Code review helps identify defects, improve maintainability, and enforce coding standards. Peer review means colleagues examine one another's work; formal review means a defined procedure with specified roles, stages, criteria, and documentation. Peer review can itself be formal, because “peer” describes who reviews while “formal” describes how structured the process is.

Differences:
• Peer review is commonly informal or semi-formal; formal review is highly structured.
• Peer review may involve one or more colleagues; formal review typically assigns moderator, author, reviewers, and recorder.
• Peer review may use pull-request comments; formal review defines preparation, meeting, findings, and closure.
• Peer review documentation may be lightweight; formal review records findings and decisions systematically.
• Peer review is efficient for routine changes; formal review suits critical, complex, or regulated modules.

Scenario: five developers build a large E-Commerce Application. Developer A handles the cart, B payment, C catalog, D orders, and E authentication/integration. The payment module is critical because defects can cause duplicate charges, incorrect status, or unauthorized transactions.

Peer-review procedure:
1. Developer B creates a focused pull request with code, tests, and a description.
2. Developers A and D review; Developer E provides security input when needed. The author is not the sole approver.
3. Reviewers examine payment amounts, currency precision, gateway requests/responses, duplicate-payment prevention, authorization, failure/timeout handling, safe logging, and test coverage.
4. Reviewers create specific comments. Example: “A gateway callback may be processed twice; use idempotency so a repeated callback cannot create a duplicate order or charge.”
5. Record each issue's location, description, severity, and required action.
6. Discuss findings respectfully, clarify misunderstandings, and agree on fixes.
7. Developer B corrects the code and adds or updates tests.
8. Reviewers recheck fixes and ensure tests pass without regression.
9. Merge only after required approvals and automated checks pass.

Example findings:
• Repeated callback may duplicate an order — Critical; add idempotency and a unique transaction constraint; test the same callback twice.
• Negative amount accepted — Major; validate server-side; test negative, zero, and valid amounts.
• Gateway exception swallowed — Major; implement error handling and reconciliation; simulate a timeout.
• Unclear method names — Minor; rename consistently and review the changes.

Benefits include early defect detection, knowledge sharing, consistent coding, less downstream testing effort, easier maintenance, and a record of decisions. Conclusion: peer review suits everyday development; structured procedures are especially valuable for payment-processing logic.

Q5. Coding-review checklist with at least 20 criteria and their importance.

A coding-review checklist is a repeatable set of questions used to examine correctness, security, performance, readability, maintainability, and compliance with development standards. Reviewers should record evidence rather than merely ticking boxes.

1. Meaningful names: descriptive class, method, and variable names improve understanding.
2. Consistent naming: agreed conventions improve consistency across the project.
3. Constant naming: clearly named constants distinguish fixed values from changing data.
4. Appropriate method length: focused methods reduce complexity and simplify tests.
5. Single responsibility: each class or method has a clear purpose and fewer unintended side effects.
6. Duplication: removing unnecessary repeated logic reduces inconsistent behavior and maintenance effort.
7. Indentation: consistent formatting improves readability.
8. Conditional clarity: understandable conditions reduce logical mistakes.
9. Correct operators: correct arithmetic and logical operators prevent wrong decisions.
10. Boundary checks: limits, empty values, and overflow cases are handled.
11. Input validation: server-side validation prevents invalid data entering the system.
12. Output encoding: context-appropriate encoding helps prevent cross-site scripting.
13. Authentication: protected functions verify identity.
14. Authorization: permission checks prevent access to another user's data.
15. Sensitive-data protection: secrets and personal data are not unnecessarily exposed.
16. SQL injection prevention: parameterized queries prevent input from altering database commands.
17. Secret management: credentials are kept out of source control.
18. Exception handling: errors are handled at suitable boundaries without hiding failures.
19. Resource management: streams, files, and database connections are reliably released.
20. Transaction management: related updates are atomic when required.
21. Concurrency safety: shared data is protected against race conditions and lost updates.
22. Performance: unnecessary loops, queries, and expensive work are avoided.
23. Database efficiency: queries fetch necessary data and use appropriate indexes.
24. Comments and documentation: comments explain important decisions and public interfaces.
25. Comment accuracy: current comments prevent misleading maintenance decisions.
26. Logging: important events and failures are recorded without exposing secrets.
27. Error messages: messages help users without revealing internal details.
28. Unit tests: normal, invalid, and boundary cases detect regressions.
29. Dependency management: approved, maintained dependencies reduce vulnerability and compatibility risks.
30. Backward compatibility: changes preserve required API/data compatibility or provide migration steps.

Use Pass, Fail, Not Applicable, or Needs Discussion for each criterion. Record the location, explanation, severity, and action for every failure. Linters can automate formatting, unused-variable, and some security checks; human reviewers must still assess business logic and design quality.

For example, reviewing a bank balance update can reveal missing amount validation, missing authorization, imprecise money types, absent transaction boundaries, poor logging, and missing concurrent-transfer tests.

Conclusion: a consistent checklist helps detect defects early and improves software quality, security, performance, and maintainability.

Q6. Equivalence Partitioning for a Student Result Management System.

Equivalence Partitioning (EP) divides input values into groups expected to produce similar behavior. The system accepts marks from 0 to 100 for five subjects and calculates total, percentage, grade, and pass/fail.

Assumptions for this example: exactly five subjects; integer marks from 0 through 100; a student passes only if every subject has at least 40 marks; percentage is total/500 × 100. Illustrative grades: A+ = 90–100, A = 80–<90, B = 70–<80, C = 60–<70, D = 50–<60, E = 40–<50, F = below 40. Actual university rules take precedence.

Input partitions:
• -5: invalid, reject.
• 0: valid, subject failed.
• 25: valid, subject failed.
• 40: valid, subject passed.
• 65: valid.
• 95: valid.
• 100: valid maximum.
• 105: invalid, reject.
• 45.5: reject if only integers are permitted.
• abc or @: reject as nonnumeric.
• Blank: reject as missing.

For a single-subject validation test, keep the other four subjects at valid marks such as 60. Representative cases should verify acceptance and rejection for each partition.

Calculation test cases:
• 0,0,0,0,0 → total 0, percentage 0%, Fail, Grade F.
• 40,40,40,40,40 → total 200, percentage 40%, Pass, Grade E.
• 60,60,60,60,60 → total 300, percentage 60%, Pass, Grade C.
• 80,80,80,80,80 → total 400, percentage 80%, Pass, Grade A.
• 100,100,100,100,100 → total 500, percentage 100%, Pass, Grade A+.

Pass/fail tests:
• 40,40,40,40,40 → Pass.
• 39,80,80,80,80 → Fail because one subject is below 40.
• 40,39,40,40,40 → Fail.
• 0,100,100,100,100 → total 400 and percentage 80%, but Fail because one subject failed.

Boundary values:
• Minimum: -1, 0, 1 — reject -1, accept 0 and 1.
• Pass mark: 39, 40, 41 — 39 fails a subject; 40 and 41 pass.
• Maximum: 99, 100, 101 — accept 99 and 100; reject 101.
• Grade boundaries: test percentages immediately below, at, and above 40%, 50%, 60%, 70%, 80%, and 90%. Ensure rounding rules are defined.

The test set covers valid/invalid input, data types, missing values, boundaries, calculations, grades, and the important distinction between overall percentage and passing every subject. EP reduces redundant cases; Boundary Value Analysis targets likely off-by-one defects.

Conclusion: use both techniques and align the final suite with the institution's exact grading, rounding, and missing-mark policies.

Q7. Equivalence Partitioning and Boundary Value Analysis for shopping discounts.

Discount rules: below ■1,000 → 0%; ■1,000–■4,999 → 5%; ■5,000–■9,999 → 10%; ■10,000–■49,999 → 15%; ■50,000 or above → 20%. Assume non-negative whole rupee amounts and discount calculated on the eligible purchase amount before separately defined taxes/shipping.

Equivalence partitions:
• Negative amount, e.g. -■100: reject.
• ■500: 0% discount.
• ■2,000: 5%.
• ■7,000: 10%.
• ■20,000: 15%.
• ■75,000: 20%.
• Nonnumeric text or blank: reject.
• Very large amount beyond supported limits: reject safely or follow defined business rules.

Boundary Value Analysis tests one rupee below, at, and above each threshold:
• ■999 → 0%, discount ■0.
• ■1,000 → 5%, discount ■50.
• ■1,001 → 5%, discount ■50.05.
• ■4,998 → 5%, discount ■249.90.
• ■4,999 → 5%, discount ■249.95.
• ■5,000 → 10%, discount ■500.
• ■5,001 → 10%, discount ■500.10.
• ■9,998 → 10%, discount ■999.80.
• ■9,999 → 10%, discount ■999.90.
• ■10,000 → 15%, discount ■1,500.
• ■10,001 → 15%, discount ■1,500.15.
• ■49,998 → 15%, discount ■7,499.70.
• ■49,999 → 15%, discount ■7,499.85.
• ■50,000 → 20%, discount ■10,000.
• ■50,001 → 20%, discount ■10,000.20.

Additional tests: ■0 should be accepted with no discount; -■1 should be rejected; blank and alphabetic input should be rejected; repeated requests should not duplicate orders or charges; a stale cart total should be recalculated or validated server-side.

Likely boundary defects:
1. Using amount > 1000 rather than >= 1000 means ■1,000 gets no discount.
2. Using amount <= 5000 for the 5% range gives ■5,000 the wrong rate.
3. Missing ■4,999 or another edge creates a gap.
4. Overlapping ranges make the result depend on condition order.
5. Wrong percentage calculation creates incorrect discount amounts.
6. Floating-point and rounding errors affect currency calculations.

EP selects representative values from each range; BVA concentrates on thresholds where rules change. Additional tests should cover final payable amounts, taxes, shipping, coupon stacking, and cart-specific rules.

Conclusion: the combined strategy covers all discount classes, invalid inputs, threshold comparisons, and calculation errors without testing every possible amount.

Q8. Test to Pass and Test to Fail using an ATM withdrawal system.

Test to Pass checks intended behavior under valid inputs and normal conditions. Test to Fail deliberately supplies invalid inputs or failure conditions to verify safe rejection and recovery.

Assume a correct PIN, active card, ■10,000 account balance, sufficient ATM cash, and permitted denominations/limits.

Test to Pass cases:
• Correct card and PIN → authentication succeeds.
• Withdraw ■1,000 → cash is dispensed and the account is debited by ■1,000.
• Withdraw ■100 if permitted → correct cash dispensed.
• Withdraw the configured per-transaction maximum → succeeds if all other limits are satisfied.
• Balance inquiry → correct balance displayed.
• Receipt request → receipt contains correct transaction details.
• Withdraw ■2,000 from ■10,000 → remaining balance ■8,000.
• Request a supported denomination combination → correct amount dispensed.

Test to Fail cases:
• Incorrect PIN → deny withdrawal and handle attempt according to policy.
• Repeated wrong PIN beyond the configured limit → block card/account access as specified.
• Blocked or expired card → reject.
• ■500 balance with ■1,000 requested → reject without debit.
• Zero or negative withdrawal → reject.
• Amount above transaction limit → reject.
• Unsupported denomination → reject or offer a defined alternative.
• ATM cash shortage → do not report successful dispensing.
• Network failure before authorization → fail safely without unauthorized debit.
• Network failure after authorization → reconcile outcome and avoid duplicate debit.
• Transaction timeout → enter a controlled pending/failure state and reconcile.
• Duplicate request after retry → prevent duplicate logical transaction.
• Session timeout → terminate session and clear sensitive data.
• Alphabetic or malformed amount → reject.

Important scenarios:
Insufficient balance must not result in cash or a debit. Invalid PIN and blocked-card rules must be enforced. If a bank authorizes a withdrawal but the response is lost, the ATM must not assume failure and blindly repeat a financial operation. Transaction identifiers and reconciliation should establish the actual outcome.

Test to Pass focuses on intended functionality; Test to Fail focuses on robustness, security, and recovery. Both must check the visible message and underlying account records.

Conclusion: a complete ATM strategy verifies successful withdrawals and ensures that invalid requests, security violations, timeouts, and network failures do not cause financial loss.

Q9. Data Testing strategy for a hospital appointment system.

A hospital appointment system accepts age, doctor, appointment date, and online payment. Data Testing checks validity, consistency, integrity, and security from user input through storage and processing.

1. Valid data: permitted age, active doctor, available date/time slot, valid patient contact details, and successful payment authorization should be accepted and stored correctly.

2. Invalid data: negative or out-of-policy age, unknown doctor ID, past appointment date where prohibited, invalid payment amount, or malformed contact information should be rejected.

3. Boundary data: test minimum and maximum permitted age, one unit outside each limit, the last date in the booking window, the first date outside it, earliest/latest appointment time, and payment limits. Exact limits must come from hospital policy.

4. Missing data: omit required age, doctor, date, payment method, or protected API patient identifier. The system should reject incomplete submissions with clear messages.

5. Wrong data types: age as text, unsupported date format, invalid doctor ID type, malformed amount, or an array where a single ID is expected should be rejected.

6. Duplicate data: submit registration twice, click booking repeatedly, make simultaneous bookings for one exclusive slot, repeat payment callbacks, or retry after a timeout. Uniqueness and idempotency should prevent duplicate appointments and charges.

7. Inconsistent data: appointment refers to inactive doctor; date disagrees with slot; appointment is paid while payment record says failed; or two bookings occupy one exclusive slot. Verify constraints and transaction consistency.

8. Extreme values: enormous ages, long names/notes, very large payments, dates far in the past/future, long IDs, and many concurrent requests should be handled safely.

9. Malicious/unexpected input: SQL injection strings, script payloads, Unicode sequences, oversized bodies, unexpected JSON properties, forged patient IDs, unauthorized appointment changes, and replayed payment notifications should be rejected or safely treated as data.

Lifecycle testing:
1. Patient submits data.
2. Server validates patient, doctor, date, and availability.
3. Appointment is created in the correct state.
4. Payment is processed.
5. Confirmed payment updates appointment and payment records consistently.
6. Failed or interrupted payment is handled without false confirmation.
7. Cancellation/refund records remain consistent.

Use synthetic patient data, a payment sandbox, database-constraint checks, frontend and API tests, security tests, concurrency tests, audit checks, and recovery tests. Record inputs, expected/actual results, and defect IDs.

Conclusion: Data Testing covers valid, invalid, boundary, missing, wrong-type, duplicate, inconsistent, extreme, and malicious data. It helps prevent booking conflicts, duplicate charges, privacy breaches, and inaccurate records.

Q10. State Transition Testing model for a bank account application.

State Transition Testing verifies behavior when system actions depend on the current state. The main lifecycle is New Account → Active → Suspended → Active → Closed.

Assumptions: a new account requires identity/opening checks before activation; suspicious activity may suspend an active account; authorized review may clear the concern; closure requires settlement and approval; a closed account is terminal in the normal workflow. Direct closure from Suspended is valid only if bank policy permits it.

Valid transitions:
• New Account + successful opening checks → Active.
• Active + suspicious activity detected → Suspended.
• Suspended + authorized clearance → Active.
• Active + approved closure and satisfied conditions → Closed.
• Suspended + permitted closure and satisfied conditions → Closed, only if explicitly allowed.

Invalid transitions:
• Withdraw while New Account is not active.
• Reactivate Suspended without approval.
• Withdraw while Suspended if policy prohibits it.
• Reactivate Closed through ordinary workflow.
• Deposit/withdraw through normal APIs after Closed.
• Suspend a Closed account.
• Close while required obligations remain unsettled, unless a defined pending-closure state exists.

Test cases:
ST01: New Account, valid activation evidence → Active.
ST02: New Account, withdrawal → rejected; state unchanged.
ST03: Active, suspicious-activity event → Suspended.
ST04: Suspended, restricted transfer → rejected.
ST05: Suspended, no authorized clearance → remains Suspended.
ST06: Suspended, authorized clearance → Active.
ST07: Active, closure prerequisites satisfied → Closed.
ST08: Active, unsettled obligations → closure rejected or pending according to specification.
ST09: Closed, withdrawal → rejected.
ST10: Closed, ordinary reactivation → rejected.
ST11: repeated suspension event → no duplicate side effects.
ST12: stale clearance event after closure → remains Closed; event rejected or safely ignored.

State coverage visits every defined state; transition coverage exercises every valid transition. Both are necessary: reaching Suspended does not prove that authorized reactivation works.

Also verify that state changes record actor, timestamp, reason, and reference; concurrent events do not create inconsistent states; operations are atomic where needed; and server-side authorization controls transitions.

Conclusion: state transition tests verify the permitted lifecycle, reject illegal operations, and protect account integrity and security.

Q11. State-based testing strategy for a Food Delivery Application.

The normal order flow is Order Placed → Payment Confirmed → Restaurant Accepted → Food Preparing → Out for Delivery → Delivered. Other states include Cancelled, Payment Failed, and Refund Initiated. Additional states such as Restaurant Rejected, Delivery Failed, and Refund Completed may be added if the real specification defines them.

Valid transitions:
• Order Placed + payment succeeds → Payment Confirmed.
• Order Placed + payment fails → Payment Failed.
• Payment Failed + permitted successful retry → Payment Confirmed.
• Payment Confirmed + restaurant accepts → Restaurant Accepted.
• Restaurant Accepted + preparation starts → Food Preparing.
• Food Preparing + courier collects order → Out for Delivery.
• Out for Delivery + confirmed delivery → Delivered.
• Order Placed + permitted cancellation → Cancelled.
• Eligible paid order cancellation → Cancelled and refund initiated if required.

Invalid transitions:
• Order Placed → Delivered directly.
• Payment Failed → preparation without an authorized exception.
• Payment Confirmed → Delivered without required restaurant and dispatch stages.
• Out for Delivery → Food Preparing without a supported return process.
• Delivered → ordinary pre-delivery cancellation.
• Cancelled → resume preparation without a defined reopening workflow.
• Refund Initiated → duplicate refund without checking existing transaction.

Repeated events include duplicate payment-success callbacks, repeated restaurant acceptance, duplicate delivery confirmation, repeated cancellation, and refund retries. Idempotency should prevent duplicate orders, charges, or refunds.

Unexpected events include payment success arriving after cancellation, stale restaurant events, delivery confirmation before dispatch, payment success after timeout, server restart during a transition, and loss of network after order placement. Validate event ordering and reconcile authoritative payment data.

Recovery scenarios:
• Payment timeout: query/reconcile before retrying.
• Restaurant rejection after payment: cancel and initiate refund if policy requires.
• Delivery failure: use a defined failed-delivery process; do not mark Delivered.
• Server crash during state change: recover without inconsistent order/payment records.

Test cases:
FD01 successful payment → Payment Confirmed.
FD02 failed payment → Payment Failed.
FD03 successful retry → Payment Confirmed.
FD04 restaurant acceptance → Restaurant Accepted.
FD05 preparation starts → Food Preparing.
FD06 courier collection → Out for Delivery.
FD07 delivery confirmation → Delivered.
FD08 eligible cancellation → Cancelled.
FD09 paid cancellation → refund initiated when required.
FD10 cancellation after delivery → rejected under normal rules.
FD11 duplicate payment callback → one logical payment result.
FD12 duplicate refund request → no duplicate refund.
FD13 late payment success after cancellation → reconcile and follow cancellation/refund policy.
FD14 server failure during transition → consistent recoverable state.
FD15 attempt to skip required state → rejected.

Coverage should include every state, every valid transition, invalid transitions, event ordering, repeated events, recovery, and consistency among order, payment, refund, and delivery records.

Conclusion: state-based testing prevents inconsistent order states and duplicate financial operations when asynchronous events arrive late or multiple times.

Q12. Decision Table Testing for an insurance claim system.

Decision Table Testing organizes combinations of conditions and the actions they produce. It is useful when insurance claim decisions depend on policy validity, premium payment, accident coverage, document availability, and claim amount.

Assumptions:
• Policy must be valid on the accident date.
• Premium must be paid.
• Accident type must be covered.
• Mandatory documents must be complete.
• Claim amount must be within the permitted limit.
• Failure of an eligibility condition causes rejection.
• Missing documents cause a request for additional documents.
• Exceptional or conflicting evidence may require manual review.

Conditions: C1 policy valid? C2 premium paid? C3 accident covered? C4 documents complete? C5 amount within limit?
Actions: approve, reject, request documents, or refer for review.

Decision rules:
R1: Policy invalid → reject.
R2: Policy valid, premium unpaid → reject.
R3: Policy and premium valid, accident not covered → reject.
R4: Eligibility conditions pass, documents incomplete → request documents.
R5: Eligibility conditions pass, amount exceeds limit → reject or refer according to defined policy.
R6: All conditions pass and amount within limit → approve.
R7: Conflicting records or fraud indicators → manual review where policy requires it.

Derived tests:
IC01 invalid policy → reject.
IC02 unpaid premium → reject.
IC03 excluded accident → reject.
IC04 eligible claim with missing documents → request documents.
IC05 all conditions satisfied and amount within limit → approve.
IC06 amount exceeds limit → reject/refer according to specified policy.
IC07 conflicting supporting records → manual review if required.
IC08 missing mandatory field → reject incomplete submission or request correction as specified.

Five binary conditions yield up to 2^5 = 32 combinations. Many may produce the same action, and some conditions become irrelevant once a decisive rejection condition is met. The team can reduce redundant tests by grouping equivalent combinations, but must preserve important interactions and rule precedence.

Decision tables expose missing combinations, contradictory business rules, and untested outcomes. Each rule should map to requirements and test cases. The insurer must explicitly define priority when multiple conditions fail simultaneously.

Conclusion: decision tables provide systematic coverage of eligibility, documentation, limits, and review rules and simplify maintenance when policies change.

Q13. Decision Table Testing for a login system.

The login system grants access only when all conditions are satisfied:
C1 registered username; C2 correct password; C3 account active; C4 account not locked; C5 CAPTCHA verified.
Action A1 grants access; A2 denies access.

There are 2^5 = 32 binary combinations. Only the combination where all five conditions are Yes should grant access. A minimum effective baseline uses one positive test and five negative tests that each isolate one failed condition.

Decision rules:
R1: all Yes → grant access.
R2: username unregistered → deny.
R3: password incorrect → deny.
R4: account inactive → deny.
R5: account locked → deny.
R6: CAPTCHA failed → deny.
A dash in the decision table means the other conditions do not change the denial outcome for that rule.

Test cases:
LG01: registered username, correct password, active, unlocked, valid CAPTCHA → access granted.
LG02: unregistered username → access denied.
LG03: incorrect password → access denied.
LG04: inactive account → access denied.
LG05: locked account → access denied.
LG06: failed or expired CAPTCHA → access denied.

Each negative test changes one condition while keeping other conditions valid wherever meaningful. This isolates failures and simplifies debugging. The set is a minimum baseline, not exhaustive; high-risk systems should also test combinations and interactions.

Additional security tests: empty and very long inputs, SQL injection attempts, repeated failed logins, CAPTCHA replay/expiry, session creation and invalidation, access to protected pages, rate limiting, lockout recovery, and generic error messages that do not disclose whether a username exists.

Conclusion: Decision Table Testing makes login rules explicit. Six core tests check success and independent enforcement of each mandatory condition; combination and security tests strengthen coverage.

Q14. Black-box password validation suite using EP, BVA, and negative testing.

Password rules: minimum 8 characters, maximum 20 characters, at least one uppercase letter, one lowercase letter, one digit, and one special character. A password is valid only if all rules are met. Assume “!” is an allowed special character; the actual allowed character set and Unicode counting rules must be specified.

Equivalence partitions:
• Valid: Abcdef1! → accept.
• Too short: Ab1!xyz → reject.
• Too long: 21-character password satisfying all character rules → reject.
• Missing uppercase: abcdef1! → reject.
• Missing lowercase: ABCDEF1! → reject.
• Missing digit: Abcdefgh! → reject.
• Missing special character: Abcdef12 → reject.
• Empty password → reject.
• Spaces only → reject if whitespace-only values are prohibited.
• Unsupported character → reject according to the defined policy.

Boundary Value Analysis:
• Length 7 → reject.
• Length 8 → accept if all character classes are present.
• Length 9 → accept if valid.
• Length 19 → accept if valid.
• Length 20 → accept if valid.
• Length 21 → reject.
• Length 0 or 1 → reject.
Use passwords that otherwise satisfy all character requirements to isolate the length boundary.

Negative tests:
• abcdefgh → missing uppercase, digit, special.
• ABCDEFGH → missing lowercase, digit, special.
• Abcdefgh → missing digit and special.
• Abcdef12 → missing special.
• 12345678 → missing uppercase, lowercase, special.
• !!!!!!!!! → missing uppercase, lowercase, digit.
• Blank → reject.
• Abc 123! → follow documented whitespace policy.
• Extremely long input → reject safely without server error.
• Null password in API request → reject safely.
• Malicious-looking input → treat as data, not executable code.
• Unicode password → follow defined character-counting and category rules.

A core suite should include valid passwords of lengths 8 and 20; invalid lengths 7 and 21; tests for each missing character category; empty, whitespace, unsupported-character, oversized, and null API inputs.

EP detects incorrect character-category validation. BVA detects off-by-one length errors. Negative testing detects missing-value handling, crashes, and unexpected-input weaknesses.

Security considerations: enforce rules server-side, store passwords using an appropriate salted password-hashing algorithm, never log passwords, define Unicode length semantics, and test password reset/change workflows separately.

Conclusion: the combined suite verifies valid acceptance, invalid rejection, boundary correctness, and safe handling of unusual inputs.

Q15. Role of static black-box testing and review of online shopping requirements.

Static black-box testing examines requirements, specifications, designs, and related artifacts without executing the application or relying on internal code structure. It focuses on whether documented behavior is clear, complete, consistent, and testable.

Roles:
• Detect requirement defects before coding.
• Clarify ambiguous language.
• Identify missing workflows and failure handling.
• Detect contradictory business rules.
• Improve testability with measurable acceptance criteria.
• Support test design and requirements traceability.

Sample draft requirements:
R1 Customers register using name, email, and password.
R2 Customers log in securely.
R3 Product search should be quick and easy.
R4 Customers can add, change quantities, and remove cart items.
R5 Orders above ■5,000 receive 10% discount.
R6 Customers can pay online using supported methods.
R7 Order is confirmed after checkout.
R8 Orders are delivered as quickly as possible.
R9 Customers can cancel orders.
R10 Application must be completely secure and never fail.

Ambiguous requirements:
• “Quickly and easily” lacks measurable search behavior.
• “Above ■5,000” leaves unclear whether exactly ■5,000 qualifies.
• “As quickly as possible” lacks delivery targets.
• “Can cancel” does not define eligible states or deadlines.
Improve each with explicit rules and measurable criteria.

Incomplete requirements:
• Registration omits email verification, duplicate email handling, password rules, and invalid input.
• Cart omits inventory and quantity limits.
• Payment omits failures, timeouts, duplicate requests, and refunds.
• Order confirmation lacks the required payment/order state.
• Cancellation omits refund eligibility and timing.

Potential contradictions to check against the full specification:
• One promotion gives 10% above ■5,000 while another gives 15% at the same threshold.
• Checkout confirmation may conflict with a rule requiring successful payment.
• Cancellation after dispatch may be permitted in one section and prohibited in another.
These are possible conflicts to investigate if the other rules exist; they are not explicit contradictions in the ten draft statements alone.

Unverifiable requirements:
• “Securely” lacks concrete security criteria.
• “Quickly” lacks response-time targets and workload.
• “As quickly as possible” lacks delivery targets.
• “Completely secure” and “never fail” are absolute claims that cannot be proven by finite tests.
Replace them with specific controls, measurable performance targets, availability goals, and recovery objectives.

Potential missing requirements: product details, stock reservation, taxes and shipping, payment security, order tracking, refunds, duplicate-payment prevention, privacy and retention, error handling, performance/availability, accessibility, and auditability. Confirm these with stakeholders rather than assuming every feature is mandatory.

Review checklist:
A. Clarity: unique requirement IDs; defined terms; measurable wording; one clear behavior per requirement.
B. Completeness: major workflows, valid/invalid/boundary inputs, error handling, payment/refund/inventory rules, roles and permissions.
C. Consistency: no conflicting rules; consistent pricing, tax, discount, order/payment states, and terminology.
D. Testability: objective acceptance criteria, observable results, numeric boundaries, performance conditions, test traceability.
E. Security/data: authentication, authorization, sensitive-data protection, validation, privacy, audit, duplicate-payment protection.
F. Maintenance/approval: dependencies, change traceability, assumptions, unresolved questions, stakeholder approval, documented review findings.

Review process:
1. Collect requirements, business rules, stories, and interface designs.
2. Review independently.
3. Classify findings as ambiguous, incomplete, contradictory, unverifiable, or missing.
4. Record requirement ID, evidence, impact, and proposed clarification.
5. Discuss with stakeholders.
6. Revise specifications and acceptance criteria.
7. Link requirements to test cases.
8. Obtain approval and close or explicitly accept remaining findings.
9. Begin implementation once agreed entry criteria are met.

Example improved requirement: “The system shall return the first page of search results within two seconds for at least 95% of requests under an agreed workload of 500 concurrent users, excluding client-side rendering.” Stakeholders must validate these numbers before adopting them.

Conclusion: static black-box testing improves requirement quality before coding. A structured checklist and review process establish clear, complete, consistent, and testable specifications, reduce rework, and form a reliable basis for dynamic testing.